AXIOM: A Shared Graph-Attention Controller for Scalable Multi-Agent 5G Slice Admission

Manoj Prasad Kunasegran[1], Wai Leong Pang[1*], Swee King Phang[1]

[1]School of Engineering, Faculty of Innovation and Technology, Taylor's University, Subang Jaya, 47500, Selangor, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous requests across cooperating base stations under strict per-slice guarantees. Multi-agent learning can adapt these decisions to shifting load, provided its gains survive careful evaluation. However, state-of-the-art studies rarely test learned controllers against trivial policies or on independent seeds. We present AXIOM, a multi-agent controller whose graph-attention encoder, shared by every gNodeB (gNB), feeds one decision head per slice, keeping its size fixed as clusters grow. A pre-registered protocol with both checks compares it with single-agent and independent deep Q-networks, multi-agent proximal policy optimisation (MAPPO), value decomposition, and tuned rules, under static and coupled time-varying load. At 7, 19, and 37 gNBs AXIOM keeps Ultra-Reliable Low-Latency Communication (URLLC) and enhanced Mobile Broadband (eMBB) service-level compliance above 93%, while every learned baseline stays below 84% and only a known static rule does better. At 37 gNBs it leads the best learned baseline by 35.0 points, 36.5 on 20 fresh seeds. There it serves 99.8% of URLLC demand with a backlog delay 7 to 44 times shorter than the learned baselines, using 7,782 parameters against up to 682,058. Under coupling it earns 27% to 36% more reward than MAPPO at 3 and 19 gNBs and 39% more than a static shedding rule at three. When load rises 10% to 20% past a tuned rule's conditions, AXIOM overtakes that rule by up to 8.7 reward per step. AXIOM is therefore a viable alternative for hands-off, self-optimising slice admission in next-generation 5G networks.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Neural Networks, Federated Learning, Open RAN

M. P. Kunasegran, orcid.org/0009-0005-3169-1873. W. L. Pang, orcid.org/0000-0001-8407-5648. S. K. Phang, orcid.org/0000-0002-7877-8766.

1 Introduction

Network slicing virtualises the radio access network (RAN) to serve Ultra-Reliable Low-Latency Communication (URLLC), enhanced Mobile Broadband (eMBB), and massive Machine-Type Communication (mMTC), each under its own Service Level Agreement (SLA). Our earlier simulation studies showed that deep reinforcement learning, including a deep Q-network (DQN), can protect URLLC SLAs at a single congested gNodeB (gNB) [16] and can balance load across a gNB cluster [17]. Those controllers, however, were centralised, whereas a real deployment spans many gNBs that share spectrum planning and a cluster-wide load-balance objective. Consequently, admission control across such a cluster becomes a multi-agent problem. We ask whether per-gNB agents that share a view of the cluster decide better than one central agent or than agents acting alone. Fig. 1 places this question in its wider system context.

This paper introduces AXIOM, an Adaptive Cross-cell Integrity-aware O-RAN Multi-agent slice-admission controller. Here O-RAN denotes the Open Radio Access Network, and integrity-aware means that every reported result passes checks of evaluation integrity. One encoder, shared by every gNB, feeds a separate Q-head for each slice, under centralised training and decentralised execution. The encoder is a Graph Attention Network (GAT) [38], and the per-slice heads, each a small multi-layer perceptron (MLP), carry AXIOM's advantage as the cluster grows. Three gaps found in our systematic review of 67 core studies motivate pairing AXIOM with a stricter evaluation, as Section 2 details. No reviewed study checks whether its outcome metric can score a degenerate policy as well as a correct one. In addition, none replicates a

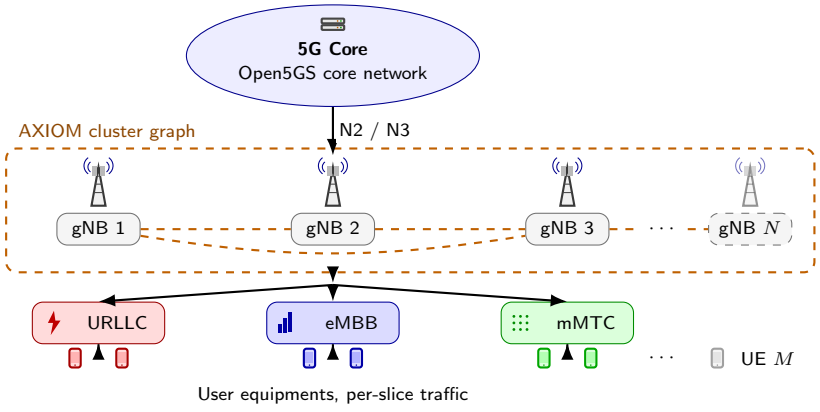


Fig. 1 Conceptual system view. A cluster of N gNBs, coordinated by the shared AXIOM encoder and backhauled to a 5G core, serves M user equipments (UEs) across three slices. AXIOM is evaluated offline, and the live rig runs one gNB

headline finding on an independent seed sample, and none tests whether a live testbed reaches the contention that its offline claims assume.

We evaluate AXIOM in two offline simulators of a gNB cluster. In Env A, with static load, shedding every mMTC request is close to optimal. Env B adds time-varying load and inter-cell coupling, so shedding one fixed slice is no longer optimal. We compare AXIOM with a single-agent deep Q-network (DQN-SA), independent per-gNB deep Q-networks (DQN-I), multi-agent proximal policy optimisation (MAPPO), and tuned threshold rules. We also grow the cluster to 7, 19, and 37 gNBs under three layouts, and test federated training, differential privacy, and disruption. The contributions of this paper are as follows.

1. AXIOM, a multi-agent controller with a shared cluster encoder and decentralised per-slice Q-heads, whose parameter count does not grow with the cluster and whose per-slice heads, rather than attention, carry its advantage at scale. In Env A it keeps priority-slice compliance above 93% at 7, 19, and 37 gNBs, where both DQN baselines and MAPPO stay below 84%. In Env B it leads DQN-SA, DQN-I, and MAPPO at three and nineteen gNBs.
2. A harder environment with time-varying load and inter-cell coupling. There AXIOM beats a static shedding rule at three gNBs, while a tuned threshold rule sets a bar that no learned policy reaches under its tuning conditions.
3. An evaluation protocol for learned slice admission control that combines trivial-policy checks, per-slice correctness metrics, independent-seed replication, tuned rule baselines, and a pre-registered analysis. It caught seven evaluation failures in this work, and fresh seeds confirmed the main findings, with the exceptions that Section 6.7 reports.
4. Clear bounds on the architecture's claims. Graph attention adds nothing over a per-node encoder, and per-step differential privacy gives only a weak guarantee.

The remainder of this paper is organised as follows. Section 2 reviews related work, Section 3 formulates the admission problem, and Section 4 presents the AXIOM architecture. Section 5 describes the experimental setup and evaluation safeguards, and Section 6 reports the results. Finally, Section 7 discusses the limitations, and Section 8 concludes the paper.

2 Related Work

Our prior systematic review [18] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own. However, the three are rarely combined under realistic constraints, and never with a validated live deployment.

Studies from the review's core sample show the state of each pillar. Learned admission control uses sequential twin actor-critics [24], digital-twin-accelerated deep reinforcement learning [34], and request forecasting to maximise the infrastructure provider's long-term revenue, as in OACR^2 [19]. AXIOM instead protects priority-slice

Table 1 Closest prior work, as described in each paper's title and abstract

Work	Problem	Multi-agent	Graph encoder	Federated
Shao et al. [30]	Slicing resource management in dense cells	Yes	Attention	Not described
Sulaiman et al. [32]	Joint slicing and slice admission in cloud RAN	Yes	Not described	Not described
Ahmadi et al. [2]	RAN and edge-computing slicing with admission control	Yes	Attention	Not described
Zangooei et al. [41]	SLA-constrained RAN slicing in Open RAN	Yes	Not described	Not described
Mohajer et al. [23]	SLA-aware task offloading and service orchestration	Yes	Attention	Not described
Bouroudi et al. [6]	Energy-efficient network-function placement	Yes	Graph neural network	Not described
AXIOM	Per-slice admission across a gNB cluster	Yes	Graph attention	Federated averaging, optional differential privacy

SLAs, the objective of an operator bound by strict per-slice guarantees. Multi-agent control spans sample-efficient multi-agent DQNs for inter-slice orchestration [8], scalable frameworks for dynamic RAN slicing [20], and cooperative agents for energy-efficient slicing [25]. A graph neural network with meta-learning makes bandwidth allocation scalable and transferable [11], and federated deep reinforcement learning has been applied to splitting virtual network functions [4] and to slice mobility [22]. Closest to our evaluation concern, Tsampazi et al. [37] compare many deep reinforcement learning agents for O-RAN in an automated pipeline, a comparison that our trivial-policy and independent-seed checks extend.

AXIOM also sits inside the broader O-RAN effort to run learned control loops over standard interfaces, which Polese et al. [27] survey and which ColO-RAN [26] and Tsampazi et al. [36] exercise on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission [5] to reinforcement learning for RAN slicing [29], and Khani et al. [13] survey it for cloud RAN. Cooperative agents usually train by value factorisation such as QMIX [28] or with a centralised critic as in MAPPO [40], and constrained multi-agent learning has been applied to Open RAN slicing at scale [41]. Recent work in this journal applies actor-critic learning [15], double deep Q-networks [12], delay-aware allocation [3], federated learning [14], and multi-agent relay selection [31] to related problems. None of these studies checks its metric against a trivial policy or replicates on independent seeds. AXIOM instead shares one encoder across agents and gives every slice its own decision head, which Section 6.3 shows matters as the cluster grows.

Several recent works come close on a single axis of this problem. Shao et al. [30] show that graph attention strengthens inter-agent cooperation, and Ahmadi et al. [2] study topology generalisation through attention. Sulaiman et al. [32] are the closest prior art to joint slicing and admission control, yet none of the three checks for degenerate policies or replicates on independent seeds. Table 1 sets AXIOM beside the six closest works, and none of them combines admission control, a graph encoder, and federated training as AXIOM does.

Robustness studies [9, 35] test disruptive conditions, as our disruption campaign does, yet neither checks whether its robustness metric is trustworthy or replicates a finding. Zhang et al. [42] and Yasin et al. [39] are the closest precedents for periodic synchronisation and for federated learning with differential privacy, but both secure the controller rather than characterise a control policy's own privacy-utility trade-off. A further set [6, 10, 23] combines two of the three pillars. Like every work above, each evaluates its own outcome metric once, on one seed or run.

3 System Model and Problem Formulation

This section formulates slice admission across a gNB cluster as a decentralised decision problem and describes the two simulators that evaluate it.

We consider a cluster of N gNBs, three in the main campaigns and up to 37 in the scaling study, each serving URLLC, eMBB, and mMTC under a shared per-gNB Physical Resource Block (PRB) budget B. At each control step every gNB holds zero or more pending requests, each tagged with a slice, and its agent accepts or rejects each one, conditioned on the cluster state. Each decision moves the slice's PRB-ratio ceiling, the control that the control-application (xApp) interface of OpenAirInterface (OAI) exposes. The binary action matches our single-gNB work, so any coordination benefit comes from the architecture. At three gNBs the cluster is treated as fully connected, a modelling choice, and the scaling study adds ring and hex-grid layouts. The model works at the PRB level and abstracts the radio channel. Channel quality changes how many PRBs a slice needs for the same traffic, and the model folds this into the per-step demand noise. A channel-aware extension that maps the signal-to-interference-plus-noise ratio to PRB demand is left to future work.

The step reward follows our earlier formulations [16, 17], with a per-slice service term net of SLA-violation penalties, a congestion charge on admitted volume, and a load-imbalance penalty. With priority weight ω_k, per-accept reward R_k, admitted count n_k, violation penalty λ_k, and an indicator $\mathbf{1}[v_k]$ of an SLA violation of slice k that step, it is

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|\mathbf{a}_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

where $\|\mathbf{a}_t\|_1$ is the total admitted this step, C_t the cluster-mean congestion level, κ the congestion coefficient, $F_t \in [0, 1]$ the min-over-max PRB-utilisation fairness ratio, and β the load-balance weight. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 5.1 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\; 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights are set so that, under congestion, the intended policy is differentiated shedding, which keeps accepting URLLC and eMBB and sheds mMTC.

Admission is therefore a decentralised partially observable Markov decision process. At step t the state s_t comprises every gNB's queues, offered demand, and slice ceilings, and agent i observes its own feature row $\mathbf{x}_i$, which the shared encoder combines with the other rows of $\mathbf{X}_t$. For each pending request it chooses $a \in \{\text{accept}, \text{reject}\}$, the shared-budget dynamics move the cluster to s_{t+1}, and every agent receives the common reward r_t of Eq. (1). The agents jointly seek a policy π that maximises the discounted return over an episode of T=60 steps,

$$\max_{\pi} \mathbb{E}_{\pi}\Big[\sum_{t=0}^{T-1} \gamma^t r_t\Big], \qquad \gamma = 0.95. \tag{3}$$

Each gNB has a budget of B=100 PRBs in both simulators. Each slice's mean offered demand is 105% of its nominal share of 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC, scaled by a fixed per-gNB multiplier between 0.9 and 1.0, denoted m_i. This puts a gNB's mean demand at about 95% to 105% of its budget, before per-step noise and backlog. When admitted demand exceeds B, the slices share it in proportion to demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog within an episode. Env B keeps all of this and adds two mechanisms. Each gNB's mean demand follows $0.95\,(1 + 0.3\sin(2\pi(s + \psi_i)/60))$ times its nominal shares, again scaled by m_i, with s the step and ψ_i a phase redrawn per gNB and episode. As a result, demand swings between about 60% and 124% of the budget. A gNB's usable budget also shrinks with its neighbours' previous load, $B_i(t) = B\,(1 - 0.15\,\bar{u}_{\mathcal{N}(i)}(t-1))$, a first-order model of inter-cell interference over the same adjacency graph the encoder reads. In Env B shedding every mMTC request leaves URLLC compliance at 62.5% at three gNBs, whereas a rule that also sheds eMBB when the URLLC queue builds keeps it at 98.9%. Per-slice compliance counts a slice as compliant only when every gNB meets it, which saturates in Env B at nineteen gNBs, so there we also report per-gNB compliance. Fig. 2 traces one control step of each simulator, and Section S10 of Online Resource 1 gives the remaining simulator details.

4 The AXIOM Architecture

This section presents the AXIOM architecture, its training rule, and its federated variant.

AXIOM differs from its DQN baselines in what each policy sees and shares. AXIOM trains one encoder over the whole cluster from every agent's experience and reads out one Q-head per slice. DQN-SA is one controller over a flattened joint state, and DQN-I gives each gNB its own learner over local features only. The two baselines are therefore ablations that remove the shared encoder.

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 3a shows the AXIOM architecture and its data flow. At every step each gNB forms a feature row $\mathbf{x}_i$ of its PRB-used ratio, congestion level, and normalised queue length, and the rows stack into $\mathbf{X} \in \mathbb{R}^{N \times d}$. A two-layer, four-head GAT over the adjacency graph maps $\mathbf{X}$ to one normalised embedding $\mathbf{z}_i$ per gNB, and Section S10 of Online

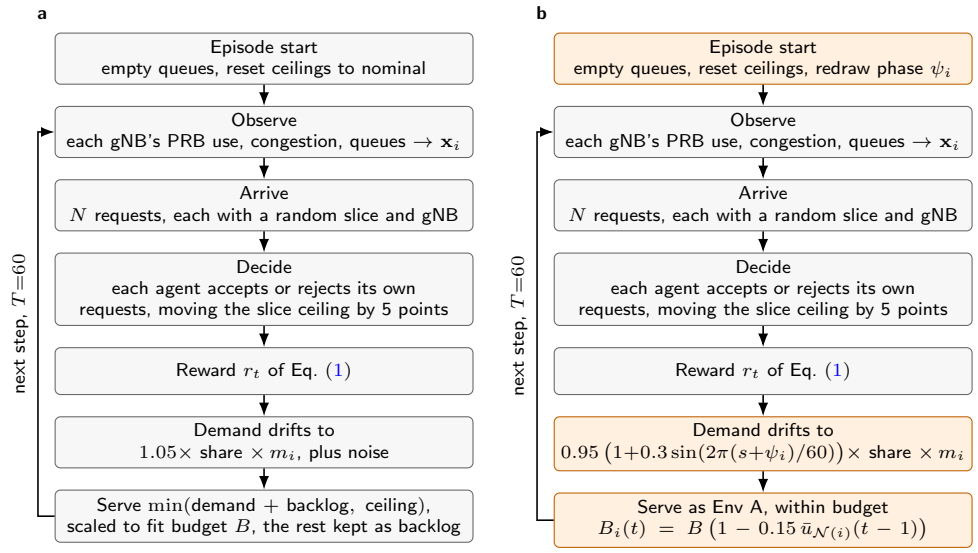


Fig. 2 One control step of each simulator, repeated for T=60 steps per episode. a, Env A with static load. b, Env B, whose three highlighted steps add time-varying load and inter-cell coupling

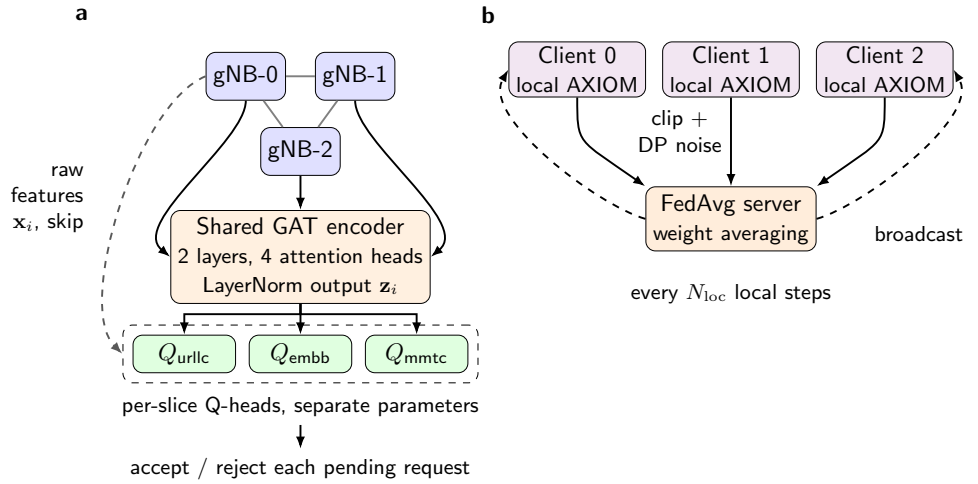


Fig. 3 AXIOM architecture. a, A shared graph-attention encoder produces one embedding per gNB, and per-slice Q-heads read that gNB's embedding and raw features. b, The federated variant, each gNB a client synchronised by periodic federated averaging with optional privacy noise

Resource 1 gives its attention equations. Every agent's loss updates this encoder, which is centralised training, while each agent decides its own pending requests from its own row $\mathbf{z}_i$, which is decentralised execution.

Algorithm 1 Centralised training and decentralised execution of AXIOM

Require: encoder f_θ, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency $\mathbf{A}$, discount γ, sync period T_s

for each step t, observing joint node features $\mathbf{X}_t$ **do**
  $\mathbf{E}_t \leftarrow f_\theta(\mathbf{X}_t, \mathbf{A})$ {centralised, one row per gNB}
  for all pending requests at gNB i, slice k **do**
    $a \leftarrow \arg\max_a Q_{\phi_k}([\mathbf{E}_t[i], \mathbf{X}_t[i]])[a]$, ϵ-greedy {decentralised, own row only}
  end for
  step env, store $(\mathbf{X}_t, \mathbf{X}_{t+1}, \{(i,k,a)\}, r_t, \text{done})$ in $\mathcal{D}$
  if $|\mathcal{D}| \geq$ warmup **then**
    sample minibatch $\mathcal{M}$, then $\mathbf{E}_\mathcal{M} \leftarrow f_\theta(\mathbf{X}_\mathcal{M}, \mathbf{A})$, $\mathbf{E}'_\mathcal{M} \leftarrow f_{\theta^-}(\mathbf{X}'_\mathcal{M}, \mathbf{A})$
    per request, $y \leftarrow r + \gamma(1-\text{done}) \max_{a'} Q_{\phi_k^-}([\mathbf{E}'_\mathcal{M}[i], \mathbf{X}'_\mathcal{M}[i]])[a']$
    $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step, then update $\theta, \{\phi_k\}$ with gradient norm clipped at 1.0
  end if
  every T_s episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for

Two design choices in AXIOM matter most at scale. One small MLP per slice gives each slice its own parameters, so URLLC's large priority weight cannot swamp mMTC's smaller learning signal in a shared head, which Section 6.3 confirms at nineteen gNBs. Each node's raw features are also concatenated onto its normalised embedding before the heads, $Q_{\phi_k}([\mathbf{z}_i, \mathbf{x}_i])$, so a head sees absolute queue and congestion levels. From here on, $\mathbf{z}_i$ denotes that concatenated input to the heads. This skip connection and a learning rate of 10^{-3}, both chosen on Env B pilot seeds before any reported run, are the only changes from the original AXIOM design.

In training, every pending request at gNB i, slice k is its own temporal-difference sample, and Algorithm 1 gives the full training loop. For a request with immediate reward r and next-state embedding $\mathbf{z}'_i$, the target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1 - \text{done}) \max_{a'} Q_{\phi_k^-}(\mathbf{z}'_i)[a'], \qquad \mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{(i,k,a)\in\mathcal{M}} \ell_\delta\Big(Q_{\phi_k}(\mathbf{z}_i)[a] - y\Big), \quad (4)$$

where $Q_{\phi_k^-}$ is slice k's target Q-head, done the episode-done flag, $\mathcal{M}$ the minibatch of requests, and ℓ_δ the Huber loss. Section 6.3 shows that a summed per-step target works as well, so the per-request scheme is a simplification rather than a source of AXIOM's advantage.

4.2 Federated Training with Differential Privacy

Fig. 3b shows the federated variant, in which each gNB holds a private copy of the network, trains only on its own requests, and every N_loc local steps sends its weights for federated averaging (FedAvg) [21]. Each client can add differential privacy through a differentially private stochastic gradient descent (DP-SGD) step [1], which clips

its minibatch gradient and adds Gaussian noise of multiplier σ. The privacy loss is accounted for with zero-concentrated differential privacy [7]. Section S10 of Online Resource 1 gives the clipping, the privacy accounting, and its assumptions.

5 Experimental Setup

This section describes the campaigns, the baselines, and the statistical tests, followed by the safeguards that protect every reported result.

Every campaign uses 300 training and 50 evaluation episodes per seed, where a seed fixes one network initialisation and one environment stream, and each seed is evaluated on a held-out stream. The three-gNB campaigns use 30 seeds, 900 to 929, with a disjoint replication sample, 1000 to 1009. The scaling campaigns use 15 seeds per layout at seven gNBs, 30 at nineteen, and at thirty-seven 30 on the hex layout and 15 on each other layout. MAPPO [40] is a published multi-agent baseline with a shared actor and a centralised critic, trained on-policy for the same 300 episodes on the same reward. Its learning rate, epochs, and minibatch size were tuned on pilot seeds, because untuned it collapsed in four of five.

Rules need no learning, and accept-all simply admits every request. The oracle rule of Env A sheds mMTC whenever its gNB's congestion level reaches a threshold τ, chosen on training streams only. The selected value is τ=0, so the rule sheds every mMTC request. We call it the oracle rule because it is the best rule of its family on training streams. It is a strong reference rather than a proven optimum, since across our Env A samples learned policies exceeded it in 28 of 450 seed-level evaluations, by at most 0.40 reward per step. In Env B we add two tuned rule families, each chosen on training streams over a grid of thresholds and evaluated on the held-out streams. The local family sheds mMTC on the gNB's own congestion or priority-slice queue and sheds eMBB on its URLLC queue, while the neighbour-aware family also reads the mean congestion of the gNB's neighbours. Two further runs repeat the DQN baselines at a learning rate of 10^{-4}.

All learned policies share $\gamma = 0.95$, the training budget, and the exploration schedule. Section S10 of Online Resource 1 lists every simulation and training parameter and the design of the sensitivity studies.

All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing. The test drops exactly tied pairs, so where two policies make identical decisions on many seeds, as AXIOM and the oracle rule do in Env A, a comparison rests on the remaining seeds.

The settings, seeds, and four hypotheses of the final campaign were fixed in a pre-registration committed to the project's version-control history before any of its runs, rather than lodged with an external registry. Every later campaign was pre-registered the same way. The Env A baselines that this campaign reuses come from an earlier rerun whose plan was committed one minute after that rerun was launched and before any of its results were read. Section S13 of Online Resource 1 lists the commit identifiers. H1 is that AXIOM beats both trivial rules in Env B, and H2 compares it with every learned baseline and tuned rule in Env B. H3 asks whether graph attention

Table 2 Evaluation failures this paper's safeguards caught during the work

Failure	Missed by	Caught by	Outcome
Inert simulator (§3)	Learned-policy metrics	Accept-all baseline	Shared-budget rerun
Never-blocking policy scored as fine	All-slice compliance	Precision, per-slice SLA	Collapse made visible
Reproduction gap (Online Resource 1)	Single-sample claim	Reproduction against replication	Churn claim retracted
Backlog carry-over (§5.1)	Per-seed means	Replication, step traces	Episode reset, rerun
Static AXIOM in Env B (§4.1)	Env A results	Env B pilot seeds	Faster learning rate, skip connection
Screen miscalibrated (§5)	Env A screen design	Pilot screen check	Pre-registered amendment
Scheduler floor (§6.8)	Cross-layer floor	Direct instrumentation	Over 100,000 readings clear

beats a per-node MLP encoder in Env B at seven gNBs, and H4 compares AXIOM with every baseline at nineteen gNBs in Env A.

A pre-registered validation screen checks every trained checkpoint on its own training stream before evaluation. Every seed is reported whether or not it passes, and Section S10 of Online Resource 1 gives the screen's criteria.

5.1 Evaluation Safeguards

Three safeguards protect the results, and Table 2 lists the failure each one caught. A trivial-policy check evaluates accept-all and one-slice shedding before any learned policy is trusted, and it exposed an earlier simulator in which accepting every request was optimal.

Correctness-aware metrics form the second safeguard of the evaluation. Block precision $\pi = b_{\mathrm{mMTC}}/b$ is the share of a policy's b blocked requests that target mMTC, and a seed that never blocks counts as a collapse rather than a success. Per-slice SLA compliance then shows whether shedding protects URLLC and eMBB, whereas all-slice compliance scores a correct policy and a failed one alike, because correct shedding gives up mMTC's SLA by design.

Finally, every headline comparison is recomputed on an independent, disjoint seed sample. This caught queue backlog carried over between episodes, and every result here comes from the corrected simulator, which empties all queues at the start of each episode.

6 Results

This section first reports static load at three gNBs and cluster-size scaling, and then the ablations, time-varying load, throughput and delay, model complexity, verification, and the wider campaigns.

We summarise per-slice compliance as priority-slice compliance, the mean of URLLC and eMBB compliance, and call a trained policy collapsed when it accepts every request. Table 3 sets out how the four learned policies stand. Throughout, a

reward margin is a difference in mean reward per step, a compliance margin is in percentage points, and a load level is a percentage of the nominal offered demand.

6.1 Static Load, Three gNBs

Table 4 reports the thirty-seed campaign in Env A. AXIOM never collapses, 99.2% of its blocks land on mMTC, and it keeps URLLC and eMBB at 99.8% and 97.9% SLA compliance. Its mean reward of 12.698 per step is 98.9% of the oracle rule's 12.835, and the two tie exactly in 22 of the 30 seeds. AXIOM earns 1.110 more reward per step than DQN-I, with 95% CI [0.60, 1.65], and 2.685 more than MAPPO, with 95% CI [2.01, 3.35], both at Holm-adjusted $p \leq 0.0002$. Its margin over DQN-SA, 0.311 with 95% CI [−0.01, 0.64], does not survive correction on thirty seeds, at $p = 0.066$. MAPPO trails both DQN baselines here, because it sheds eMBB as well as mMTC, which lets eMBB compliance fall to 57.6%.

A disjoint replication on seeds 1000 to 1009 and the pooled 40 seeds favour AXIOM more strongly, as Section S12 of Online Resource 1 reports. Nonetheless, fresh seeds in Section 6.7 do not reproduce the lead over DQN-SA.

6.2 Cluster-Size Scaling

We grow the cluster in Env A from three gNBs to seven, nineteen, and, in a pre-registered extension, thirty-seven, under fully-connected, ring, and hex-grid layouts. Every gNB keeps its own 100-PRB budget and demand, so per-gNB load stays the same as the cluster grows. DQN-I ignores the layout and gives identical results under all three, which checks that the layout changes nothing else. We test the per-layout gaps on reward and on priority-slice compliance, Holm-correcting the 18 comparisons at each size.

At seven gNBs AXIOM keeps URLLC and eMBB at 99.8% and 94.4%, against 91.4% and 61.0% for DQN-SA, 94.2% and 55.0% for DQN-I, and 87.0% and 48.4%

Table 3 How the four learned policies compare. Env A has static load, Env B time-varying load with inter-cell coupling. Priority SLA is the mean of URLLC and eMBB compliance

Property	AXIOM	DQN-SA	DQN-I	MAPPO
Design	Shared encoder, per-slice heads	One flattened controller	One learner per gNB	Shared actor, central critic
Parameters at execution	7,782 at any size	21,250 to 64,770	55,302 to 682,058	5,122
Env A reward, 3 gNBs	12.70, 98.9% of oracle	12.39	11.59	10.01
Env A priority SLA, 7, 19, 37 gNBs	97.1, 97.3, 93.7%	76.2, 65.3, 58.7%	74.6, 56.4, 32.1%	67.7, 60.6, 56.7%
Collapse, Env A, 19 and 37 gNBs	1/90 and 6/60	84/90 and 26/30	3/90 and 3/30	0/90 and 0/30
Env B reward, 3 and 19 gNBs	8.83 and 80.28	7.49 and 74.74	7.79 and 71.20	6.97 and 59.07
Env B priority SLA, 7 gNBs	35.1%	44.3%	39.9%	35.9%
Federated, noise-free	Cost +0.49 [−0.08, 1.09]	Not applicable	Not applicable	Not applicable

Table 4 Env A, three gNBs, 30 seeds per policy, 95% bootstrap CIs. Precision averages the seeds that block at all. SLA columns give per-slice compliance, and LR is the learning rate

Policy	Reward/step	Precision	Collapsed	URLLC	eMBB
Oracle rule	12.835 [12.77, 12.91]	1.000	0/30	1.000	1.000
AXIOM	12.698 [12.49, 12.86]	0.992	0/30	0.998	0.979
DQN-SA	12.387 [12.11, 12.63]	0.990	0/30	0.994	0.941
at LR 10^{-4}	11.477 [10.61, 12.09]	0.944	0/30	0.979	0.866
DQN-I	11.587 [11.11, 12.02]	0.962	0/30	0.993	0.865
at LR 10^{-4}	11.119 [10.73, 11.50]	0.878	0/30	0.985	0.794
MAPPO	10.013 [9.30, 10.73]	0.707	2/30	0.978	0.576

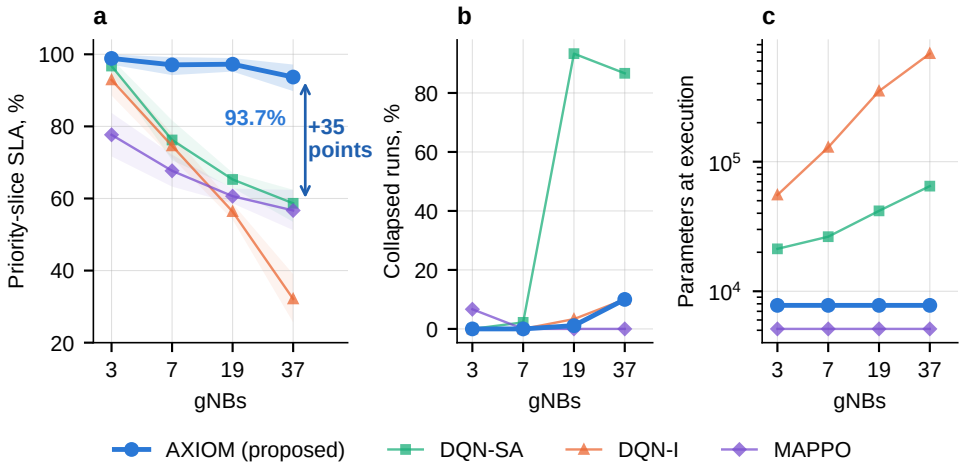


Fig. 4 Env A as the cluster grows, all layouts pooled, with the layout-independent baselines run once per seed at 37 gNBs. a, Priority-slice compliance, mean and 95% CI. b, Share of trained policies that collapse into accept-all. c, Parameters used at execution

for MAPPO. On fresh seeds the two DQN baselines reach up to 83.3%, as Section 6.7 reports. Under every layout it keeps 17.9 to 29.9 percentage points more priority-slice compliance than each baseline and earns 3.40 to 7.07 more reward per step. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.011$.

At nineteen gNBs the gaps widen, and they answer hypothesis H4. AXIOM keeps URLLC and eMBB at 99.4% and 95.1%, a pooled priority-slice compliance of 97.3% with 95% CI [95.2%, 99.0%]. DQN-SA, DQN-I, and MAPPO keep 65.3%, 56.4%, and 60.6%. Under every layout AXIOM keeps 31.0 to 42.1 percentage points more compliance than every baseline and earns 10.7 to 18.6 more reward per step, winning 28 to 30 of 30 seeds. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0001$. Fig. 4b shows DQN-SA collapsing into accepting every request in 84 of its 90 runs, against 3 for DQN-I, 1 for AXIOM, and none for MAPPO.

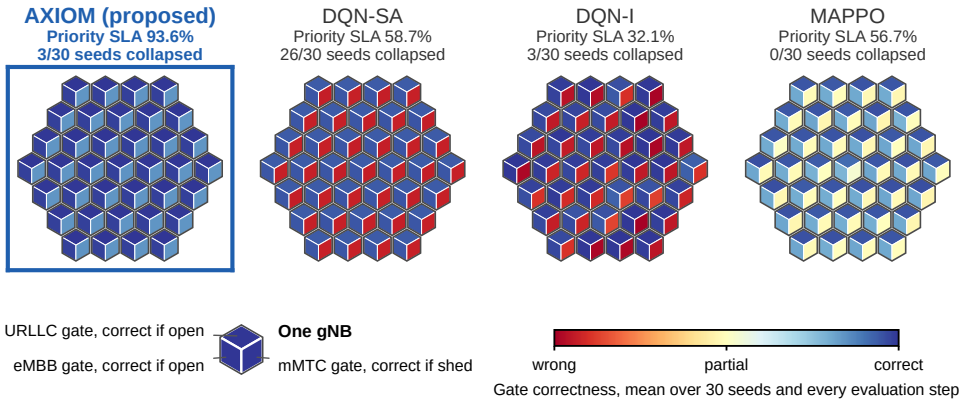


Fig. 5 Decision map of the 37-gNB hex cluster in Env A over 30 seeds. Each hexagon is a gNB at its true position, split into its URLLC, eMBB, and mMTC admission gates and coloured by how correct each gate is

At thirty-seven gNBs the lead grows further, in an extension pre-registered before any of its runs. The configuration scales the arrivals exactly as before, so each gNB still sees 1.15 pending requests per step, and no setting was tuned for this size. AXIOM runs 30 seeds on the hex layout and 15 on each of the other two. The three baselines never read the layout in Env A, so each runs once on the 30 hex seeds. AXIOM keeps URLLC and eMBB at 96.7% and 90.7%, a pooled priority-slice compliance of 93.7% with 95% CI [89.8%, 97.2%]. DQN-SA, MAPPO, and DQN-I keep 58.7%, 56.7%, and 32.1%. Under every layout AXIOM keeps 32.0 to 67.8 percentage points more compliance than every baseline and earns 18.3 to 29.5 more reward per step. Here all 18 comparisons hold after Holm correction within their pre-registered families, at adjusted $p < 0.01$. AXIOM collapses in 6 of its 60 runs, more often than at nineteen gNBs, against 26 of 30 for DQN-SA. Even its collapsed runs keep 59% to 65%, the level of the best baselines.

A replication on 20 fresh seeds, also pre-registered, confirms the result. There AXIOM keeps 95.1% priority-slice compliance against 58.5%, 57.4%, and 34.1% for MAPPO, DQN-SA, and DQN-I, and it beats each of them on compliance and on reward at Holm-adjusted $p < 0.001$. It collapses in 1 of the 20 seeds, against 18 for DQN-SA. Section S15 of Online Resource 1 gives the setup and every policy's results.

Accepting every request is the natural floor at scale. It keeps 70.9%, 67.1%, and 61.9% priority-slice compliance at seven, nineteen, and thirty-seven gNBs, and at nineteen and thirty-seven gNBs every baseline falls below it on average. Their runs that do not collapse reject priority requests as well as mMTC. At thirty-seven gNBs those runs of DQN-SA and DQN-I keep only 35.9% and 29.1%, so collapsing into accept-all is not the worst outcome for them. AXIOM stays 26 to 32 points above the floor. Its advantage at scale therefore lies in reliably learning the correct shedding policy, the one the static rule encodes, rather than in coordination beyond that rule. The baselines fail to learn this policy as the cluster grows.

Furthermore, tuning the DQN baselines does not close the gap. A pre-registered grid over their learning rate and minibatch size, chosen on pilot seeds, selected new settings in three of the four combinations of baseline and size at nineteen and thirty-seven gNBs. Tuned, DQN-I rises from 56.4% to 59.1% at nineteen gNBs and from 32.1% to 45.5% at thirty-seven. The tuned DQN-SA at nineteen gNBs accepts every request in all 30 seeds and so matches accept-all exactly. No tuned baseline exceeds the accept-all floor, and AXIOM beats each of them on compliance and on reward at Holm-adjusted $p < 0.001$, by 29.9 to 48.1 points.

A published baseline, the value-decomposition network (VDN) [33], trains DQN-I's per-gNB networks on one joint summed target. It ties AXIOM at three gNBs, at 12.664 against 12.698 reward per step in Env A and 8.552 against 8.832 in Env B, but at nineteen gNBs it keeps 83.4% priority-slice compliance. There AXIOM keeps 13.6 percentage points more and earns 2.73 more reward per step, winning 27 to 28 of 30 seeds at Holm-adjusted $p < 0.001$. Similarly, harder contention leaves this ranking of the learned policies intact. At 110% and 120% of the nominal load at nineteen gNBs, AXIOM keeps 63.4% and 21.7% priority-slice compliance, 24.3 to 34.0 and 2.5 to 6.4 percentage points more than every learned baseline, at Holm-adjusted $p \leq 0.006$. At 120% the priority demand exceeds the slice caps at most gNBs, so no policy can protect it throughout.

Fig. 4a traces the same result across sizes. AXIOM's priority-slice compliance stays between 97.1% and 98.9% from three to nineteen gNBs while every baseline falls. Its lead over the best baseline grows to 32.0 points at nineteen gNBs, with a parameter count that does not change. At thirty-seven gNBs every baseline falls again and the lead reaches 35.0 points. The static rule that sheds every mMTC request keeps 99.96% to 99.99% at every size. Where that rule is known, AXIOM comes within 3 points of it up to nineteen gNBs and within 6.3 at thirty-seven, while from seven gNBs up every other learner falls at least 24 points short.

Fig. 5 shows where that gap comes from, gNB by gNB, in the thirty-seven-gNB hex cluster. A gNB's admission gate for a slice is the PRB-ratio ceiling that its accept and reject decisions move between the slice's floor and cap. We average its openness over every evaluation step of all 30 seeds. AXIOM keeps the URLLC and eMBB gates open at every gNB, at 1.00 and 0.98, while closing the mMTC gate to 0.16, a mean that includes its three collapsed seeds. AXIOM thus sheds the right slice everywhere in the cluster. DQN-SA leaves every gate open at 0.93 to 0.95, DQN-I closes the mMTC gate only to 0.94, and MAPPO partly closes the eMBB gate, to 0.80, while leaving mMTC open at 0.51. The nineteen-gNB cluster gives the same picture, as Section S12 of Online Resource 1 reports.

Fig. 6 summarises how the four learned policies compare. AXIOM sits highest on both axes of panel a, accuracy at three gNBs and priority-slice protection at thirty-seven, and it leads on four of the six axes of panel b. MAPPO, which never collapses at thirty-seven gNBs, leads on collapse resistance, and DQN-SA leads on Env B compliance at seven gNBs.

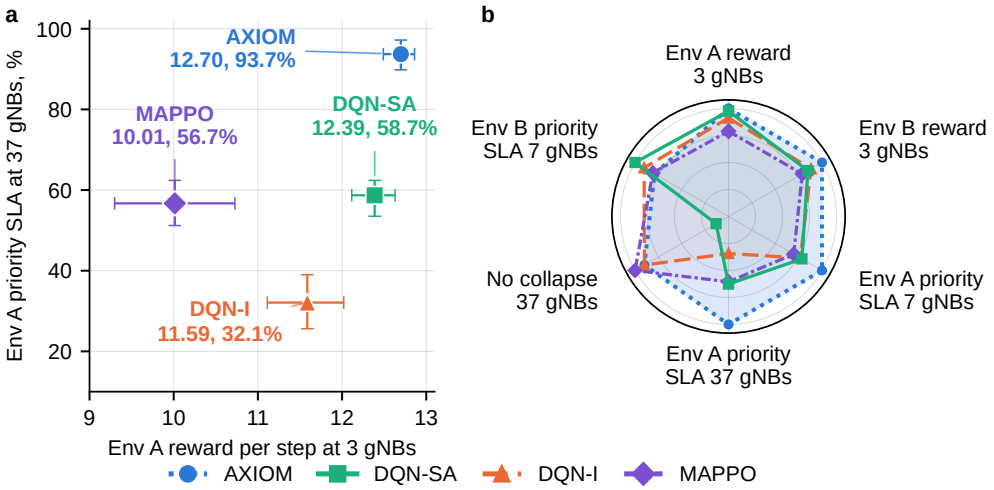


Fig. 6 AXIOM against the three learned baselines. a, Env A reward per step at three gNBs against compliance at 37 gNBs, with 95% CIs. b, Six measured axes, each as a fraction of the best learned policy on it

6.3 Ablations and Sensitivity

Four checks test where AXIOM's behaviour comes from. First, the learning rate matters for AXIOM and its baselines alike. In Env B the original AXIOM, at 10^{-4}, earns 2.37 less per step than the current design in all ten seeds, while a faster rate or a wider encoder changes nothing significant. Section S12 of Online Resource 1 gives the full study.

Second, graph attention adds nothing measurable, so a per-node MLP encoder could replace it. In Env A that variant ties AXIOM, which earns 0.058 more reward per step at three gNBs with 95% CI $[-0.17, 0.26]$ and 0.31 more at nineteen gNBs on the ring layout, with 7 of 12 seeds tied. In Env B at seven gNBs, the pre-registered test H3, it keeps 1.12 and 0.95 more reward per step on the ring and hex layouts, winning 10 of 15 seeds in each, at Holm-adjusted $p = 0.061$ and $p = 0.095$. On ten fresh seeds its lead was significant. AXIOM's gains therefore come from its shared encoder, per-slice heads, and joint training rather than from attention. AXIOM's results at nineteen and thirty-seven gNBs all use the attention encoder, and Section S12 of Online Resource 1 lists the variant's results.

Third, AXIOM's behaviour holds in Env A when the environment changes. Under 102% and 107% load and changed slice weights it stays within 0.13 of the oracle rule's reward with no significant difference. Section S12 of Online Resource 1 gives its comparisons with the baselines.

Fourth, the per-slice Q-heads matter as the cluster grows. A variant with one shared head of matched size ties AXIOM at three gNBs in both environments. At nineteen gNBs on the ring layout, however, it collapses in 9 of 15 seeds and keeps only 65.9% priority-slice compliance against AXIOM's 96.8% on the same seeds, losing every seed

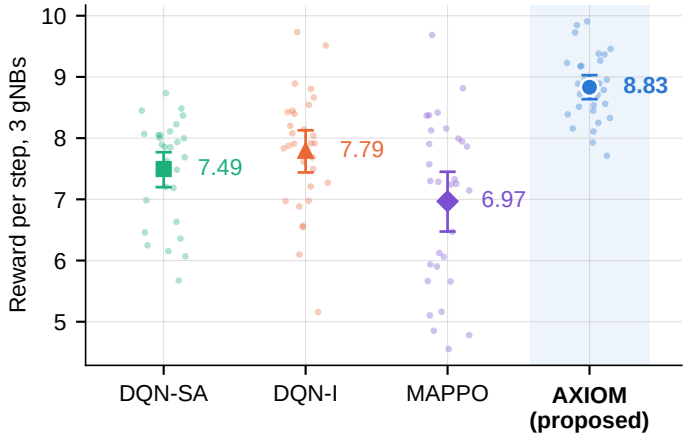


Fig. 7 Env B reward per step of the learned policies at three gNBs, 30 seeds, with per-seed points behind the mean and 95% CI

at Holm-adjusted $p < 0.001$. Training on one temporal-difference sample per request, by contrast, is not what makes AXIOM work. A variant that instead regresses the sum of a step's chosen values on one joint target, as value decomposition does, is never significantly different and trends slightly better. It reaches 12.831 reward per step in Env A at three gNBs and 99.96% priority-slice compliance at nineteen. Because this was found after the reported campaigns, we tested it once on 60 fresh seeds across four settings. It was again never significantly different from AXIOM, ahead in three settings and behind at seven gNBs in Env B, so we keep the reported design.

The comparison also holds when the baselines are made stronger. A grid of 24 MAPPO configurations, chosen on pilot seeds and allowing three times the training budget, kept the reported setting in Env B and selected a longer-trained one in Env A. There MAPPO rises from 10.013 to 11.448 reward per step at three gNBs and from 60.6% to 69.6% priority-slice compliance at nineteen gNBs on the ring layout. AXIOM still beats it in every setting tested, by 1.25 to 16.18 reward per step at Holm-adjusted $p < 0.001$, and by 2.42 in Env B at seven gNBs. At thirty-seven gNBs, in a check reported but not predicted, the selected MAPPO keeps 65.6% priority-slice compliance. AXIOM keeps 29.2 percentage points more compliance and earns 23.23 more reward per step on 15 seeds, at unadjusted $p < 0.001$. With its initialisation properly seeded, DQN-SA keeps 80.3% and 63.9% priority-slice compliance at seven and nineteen gNBs, and AXIOM beats it under every layout at Holm-adjusted $p \leq 0.035$.

6.4 Time-Varying Load with Inter-Cell Coupling

Env B tests the case a static benchmark cannot, where shedding one fixed slice is no longer optimal. Table 5 and Fig. 7 report the thirty-seed campaign at three gNBs, and fifteen seeds per layout run at seven. First, adapting to the load pays off clearly. Shedding every mMTC request, the optimum of Env A, now leaves URLLC compliance

at 62.5%. AXIOM earns 2.495 more reward per step than that rule and 4.476 more than accept-all, winning all 30 seeds against each at Holm-adjusted $p < 0.0001$, as hypothesis H1 predicted. AXIOM keeps URLLC at 95.8% by shedding eMBB as well as mMTC when URLLC queues build.

Second, among learned policies AXIOM leads at three gNBs. It earns 1.038, 1.337, and 1.863 more reward per step than DQN-I, DQN-SA, and MAPPO, and keeps more priority-slice compliance too, winning 24 to 29 of 30 seeds, all at Holm-adjusted $p < 0.0001$. The disjoint ten-seed replication points the same way, with AXIOM winning 7 or 8 of 10 seeds against each, but no lead survives correction there.

Third, a tuned rule beats every learned policy. The best local rule sheds every mMTC request and sheds eMBB whenever the URLLC queue passes 0.2. It earns 9.252 per step, 0.420 more than AXIOM, winning 22 of 30 seeds at $p < 0.0001$. The neighbour-aware rule earns slightly less, at 9.041, so neighbour information did not help the rules either. The rule encodes domain knowledge of which slice to shed under which pressure, which AXIOM must learn from the reward alone in 300 episodes. No learned policy or tuned rule keeps eMBB compliance above 37% in Env B, and even the best rule chosen in hindsight on each evaluation stream reaches only 37.7%, as Section S14 of Online Resource 1 shows.

However, at seven gNBs every learned policy falls further behind. The tuned rules earn 26.5 to 27.3 per step and beat AXIOM in all 15 seeds under every layout, while AXIOM earns 23.4 to 23.6, no more than the static rule's 23.6. Its only significant leads over a learned baseline at this size are over MAPPO's reward, and on the ring layout its compliance is below DQN-SA's, at Holm-adjusted $p = 0.011$.

A pre-registered run at nineteen gNBs on the hex layout, with 15 seeds, extends this comparison. Cluster-level compliance saturates there, at 16.7% to 24.7% for every learned policy. We also measure per-gNB compliance, the share of steps and gNBs at which a slice meets its SLA under the framework's own per-gNB test. AXIOM earns 5.54 to 21.22 more reward per step than every learned baseline and keeps 5.5 to 10.9 percentage points more per-gNB priority-slice compliance, at Holm-adjusted $p \leq 0.013$, keeping 77.3%. It does not beat the static rule, which earns 1.31 more per step, and in 9 of the 15 seeds it makes exactly the static rule's decisions. The tuned local rule, carried over from seven gNBs without re-tuning, earns 3.81 more

Table 5 Env B, three gNBs, 30 seeds per policy, 95% bootstrap CIs. Rules above the line, learned policies below it

Policy	Reward/step	URLLC	eMBB	mMTC
Accept all	4.356 [4.13, 4.60]	0.446	0.265	0.955
Shed all mMTC	6.337 [6.00, 6.67]	0.625	0.367	0.371
Tuned local rule	9.252 [9.12, 9.40]	0.989	0.367	0.390
Tuned neighbour-aware rule	9.041 [8.93, 9.15]	0.988	0.283	0.901
AXIOM	8.832 [8.64, 9.03]	0.958	0.344	0.406
DQN-SA	7.495 [7.20, 7.77]	0.866	0.284	0.412
DQN-I	7.794 [7.44, 8.13]	0.910	0.270	0.441
MAPPO	6.968 [6.47, 7.45]	0.797	0.289	0.579

than AXIOM and keeps 84.5% per-gNB compliance, winning every seed. Section S16 of Online Resource 1 lists every policy.

That lead depends on the conditions matching those the rule was tuned for. When the load rises after tuning and training, with neither the rule re-tuned nor AXIOM retrained, AXIOM overtakes the rule. At three gNBs this test used 25 checkpoints not used to find the effect. At 110%, 115%, and 120% of the nominal load AXIOM earns 0.46, 0.43, and 0.35 more reward per step than the nominally tuned rule, winning 21 to 23 of 25 seeds at Holm-adjusted $p \leq 0.0002$. At seven gNBs on the ring layout the reversal is larger. AXIOM trails the rule by 3.84 reward per step under nominal load but leads it by 1.86 and 3.14 at 110% and 120% load. A rule re-tuned on the shifted conditions draws level with AXIOM at 110% and 115% load and beats it by 0.62 reward per step at 120%. Under lighter load or stronger coupling the tuned rule keeps a lead that is not significant.

The overload result also holds at nineteen gNBs, for the same 15 checkpoints and the rule carried over from seven gNBs. At 110%, 115%, and 120% of the nominal load AXIOM earns 6.23, 7.55, and 8.67 more reward per step than that rule, winning 14 of 15 seeds at each level at Holm-adjusted $p < 0.001$. The rule keeps more per-gNB compliance by shedding eMBB. Meanwhile the static rule earns 1.8 to 2.0 more reward than AXIOM, which makes the static rule's decisions in 9 to 11 of the 15 seeds. When per-gNB load differs widely instead, with multipliers drawn between 75% and 125% at seven gNBs, AXIOM does not beat the nominally tuned rule, earning 0.36 more reward per step at $p = 0.23$. It ties a rule re-tuned to these conditions, while the static rule earns 0.27 more at $p = 0.028$. It still beats every learned baseline there, by 1.18 to 4.28 reward per step at unadjusted $p \leq 0.005$. Charging the SLA penalty per violating gNB, so that it grows with the cluster, did not raise AXIOM's per-gNB compliance, at 79.2% against 77.3% with $p = 0.65$. AXIOM still did not beat the static rule. Under that reward every learned baseline kept 13.7 to 18.5 percentage points less per-gNB compliance than AXIOM, in every seed. Section S17 of Online Resource 1 lists every policy in these runs.

With three times the training budget AXIOM also comes within reach of the tuned rule under nominal conditions, as Section S14 of Online Resource 1 reports.

6.5 Throughput and Delay

Re-running the stored evaluations with a recorder, which reproduced every stored log that could be checked, gives each slice's throughput and a backlog-based delay. The delay is the mean backlog divided by the mean served PRBs, in control periods of 5 s, a fluid-model proxy rather than packet latency. At thirty-seven gNBs in Env A, AXIOM serves 99.8% of the offered URLLC demand and 99.0% of the eMBB demand, against 94.2% to 97.7% and 85.7% to 97.5% for the learned baselines. Its URLLC delay is 0.08 control periods with a 95th percentile of 0.42, whereas the baselines reach 0.58 to 3.61 with 95th percentiles of 5.2 to 16.9, and the static rule reaches 0.03. In Env B at nineteen gNBs, AXIOM's URLLC delay of 1.35 control periods is below every learned baseline's 2.17 to 2.66, while the tuned rule's is lower still, at 0.51. Section S18 of Online Resource 1 lists every setting.

6.6 Model Complexity

Table 6 sets out the cost of each policy at every cluster size. AXIOM's shared encoder keeps its parameter count at 7,782 at every cluster size, while the DQN baselines grow with it. AXIOM's greedy decisions take 1.0 to 9.9 ms per control step on one CPU thread, well inside the 5 s control period of the live testbed. Section S12 of Online Resource 1 shows the training curves, in which MAPPO rises fastest but levels off lowest. At nineteen gNBs AXIOM is still improving at the end of its 300 episodes, while the DQN baselines have settled.

MAPPO is also the cheapest learner to run, so it is the natural low-cost alternative to AXIOM. At thirty-seven gNBs it uses 5,122 parameters, 0.27 ms per control step, and 495 s of training per seed, against AXIOM's 7,782 parameters, 9.89 ms, and 1,182 s. The extra cost buys the most where the environment is dynamic. In Env B, with time-varying load and inter-cell coupling, AXIOM earns 27% more reward per step than MAPPO at three gNBs, 8.83 against 6.97, and 36% more at nineteen gNBs, 80.28 against 59.07. Under uneven per-gNB load at seven gNBs it earns 27% more and wins every seed. At seven gNBs it earns between 7% and 15% more, significantly on two of the three layouts. A 24-configuration tuning grid for MAPPO did not close the gap in any setting where it was tested. AXIOM's slowest decision takes 0.2% of the 5 s control period, so its inference cost does not constrain deployment. Its training cost, by contrast, is paid only once and offline.

6.7 Reproducibility and Fresh-Seed Verification

We checked the results in three ways, with every pass criterion fixed before the run it judged. First, rerunning eleven reported settings in fresh processes reproduced every stored result exactly. The one exception is DQN-SA, which builds its network before setting the seed, and seeding it leaves AXIOM's leads unchanged.

Second, four further runs retrained every policy on fresh seeds, 10 to 40 per setting, and 81 of their 90 pre-stated checks held. At seven and nineteen gNBs in Env A AXIOM kept 93.4% to 100% priority-slice compliance and beat every baseline under every layout. This held at Holm-adjusted $p \leq 0.032$ in all but one ten-seed setting, which a twenty-seed rerun then confirmed at $p \leq 0.007$. In Env B at three gNBs it again beat the static rule and every learned baseline at Holm-adjusted $p \leq 0.0002$, and the tuned rules again beat every learned policy. The disruption, federation, and privacy results also held as reported. Online Resource 1 reports every number from these runs.

Table 6 Model complexity at 3, 7, 19, and 37 gNBs. Parameters used at execution, greedy inference time per control step on one CPU thread, and median training time per seed at 3, 19, and 37 gNBs, on the ring layout for AXIOM

Policy	Parameters	ms per step	Training s
AXIOM	7,782 at every size	1.04 / 1.98 / 4.79 / 9.89	624 / 892 / 1182
DQN-SA	21,250 / 26,370 / 41,730 / 64,770	0.61 / 1.42 / 3.95 / 8.02	85 / 202 / 358
DQN-I	55,302 / 129,038 / 350,246 / 682,058	0.59 / 1.38 / 3.73 / 8.08	181 / 1054 / 2088
MAPPO	5,122 at every size	0.19 / 0.20 / 0.22 / 0.27	73 / 256 / 495

The nine checks that failed concern secondary margins rather than any headline result. On fresh seeds the DQN baselines reached up to 83.3% compliance at seven gNBs, above the 80% bound we had set. AXIOM's small lead over DQN-SA in Env A at three gNBs did not recur, at margins of -0.021 and 0.159 reward per step on 20 and 40 new seeds. At 102% load that baseline also matched AXIOM. Two weak AXIOM seeds also left one ten-seed load sample 3.2% short of the oracle, a gap that a twenty-seed rerun did not reproduce. A fifth fresh-seed run, at thirty-seven gNBs on 20 seeds, held all three of its checks, as Section 6.2 reports.

Third, a script written independently of the analysis pipeline, with its own log reader and Holm correction, recomputes the numbers in every results table and the key results sentences from the raw logs. In the current version, all 666 of its checks pass.

6.8 Federation, Privacy, Disruption, and Live Deployment

Three further campaigns bound AXIOM's wider claims, and Sections S9 and S12 of Online Resource 1 report them in full. In federated training each gNB trains its own copy of AXIOM on its own requests, and the copies are averaged periodically. Pooled over all 30 federated seeds, including ten fresh ones, the cost is $+0.491$ per step with 95% CI $[-0.08, 1.09]$ and $p = 0.14$, so federation costs little on average. Even so, a cost of up to about one reward per step cannot be excluded. Adding DP-SGD noise to every step lowers reward by 0.77 to 1.25 per step, and no drop survives Holm correction at ten seeds. The formal guarantee is weak, with ε of about 2,600 or more, so per-step DP-SGD delivers no meaningful guarantee here.

Under gNB dropout and agent churn every policy loses reward significantly, and no gap between AXIOM and any baseline survives Holm correction at ten seeds. Demand spikes leave AXIOM's decisions largely intact, with block precision at 0.98, and Section S12 of Online Resource 1 gives the protocol.

A live single-gNB OAI testbed confirmed the control path that AXIOM would use, with 0.000% loss at one and at two UEs per slice. It never ran short of capacity, so it cannot show admission under contention, and AXIOM itself is evaluated offline, as Section S9 of Online Resource 1 reports.

7 Discussion and Limitations

No learned baseline earns significantly more reward than AXIOM in any setting we tested, and its margin over the cheaper MAPPO is largest under time-varying load, coupling, and uneven per-gNB load. Against hand-built rules, however, the picture is much narrower. A rule known in advance matches or beats AXIOM in steady conditions at every size, and AXIOM beats a tuned rule only when the load moves beyond the rule's tuning conditions. Its practical case is for operators who cannot hand-tune and re-tune rules as conditions change, rather than for a fixed, well-understood cluster.

Several limitations bound the findings reported in this paper. First, both simulators are synthetic, with a first-order coupling model, no radio channel, no user mobility, and no measured traffic traces, and the live testbed never reached contention. External validity on hardware under load therefore remains the main open question. Second, we did not reimplement a published slice-admission algorithm, so our learned baselines are

standard multi-agent methods and ablations of AXIOM rather than prior admission controllers. Third, attention added nothing over a per-node encoder, and a value-decomposition baseline matched AXIOM at three gNBs, so AXIOM's contribution is a design that scales rather than a new learning principle. Fourth, charging the SLA penalty per violating gNB, so that it grows with the cluster, changed neither AXIOM's compliance nor its standing against the static rule. Finally, the pre-registrations were committed to the authors' own version-control history rather than an external registry, and development work that changed no reported result is disclosed in Section S13 of Online Resource 1.

8 Conclusion

This paper presented AXIOM, a multi-agent slice admission controller with a shared graph-attention encoder and per-slice heads, evaluated under a protocol designed to make its results trustworthy.

AXIOM's advantage over learned controllers grows with the cluster. Under static load it reaches 98.9% of an oracle rule's reward at three gNBs and keeps priority-slice compliance between 93.7% and 97.3% from seven to thirty-seven gNBs, while every learned baseline stays below 84%. At thirty-seven gNBs its lead over the best learned baseline reaches 35.0 points, and 36.5 points on 20 fresh seeds, with 7,782 parameters against up to 682,058. There it serves 99.8% of URLLC demand at a mean URLLC delay of 0.08 control periods, against 0.58 to 3.61 for the learned baselines. Tuned DQN baselines stay 29.9 to 48.1 points behind it, and a value-decomposition baseline that ties it at three gNBs trails by 13.6 points at nineteen.

Under time-varying load and coupling AXIOM earns 27% to 36% more reward than MAPPO at three and nineteen gNBs, so its extra cost pays most where the environment is dynamic. When load rises 10% to 20% above the conditions a tuned rule was set for, AXIOM overtakes that rule at nineteen gNBs by 6.2 to 8.7 reward per step without retraining. Simple rules still set the bar in other settings. A static rule known in advance does better under steady load and at nineteen gNBs under coupling, and a tuned rule beats every learned policy at its tuning conditions, as Section 7 discusses. The main findings held on fresh seeds never used before.

More broadly, the evaluation safeguards show that learned admission control should be judged against trivial and tuned rules, with metrics tied to the objective, on independent seeds, under a pre-registered analysis.

Three directions for future work follow from these limits. First, a live multi-gNB deployment of AXIOM would test its gains under real contention. Second, giving the learner the queue-threshold structure that the tuned rules exploit may close the remaining gap to them. Finally, adding noise to the per-round model updates rather than to every step would make the formal privacy guarantee meaningful.

Supplementary information. Online Resource 1 is a PDF that gives the full verification results, the live testbed check, additional setup details, and additional results.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Statements and Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- **Funding:** This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.
- **Competing interests:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- **Consent for publication:** Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- **Data availability:** The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request. They are not yet publicly archived because the research is ongoing and the dataset is not yet complete.
- **Materials availability:** Not applicable, since this study involves no physical or biological materials.
- **Code availability:** The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- **Author contributions:** M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, et al (2016) Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Ahmadi M, Moayyedi A, Sulaiman M, et al (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. IEEE Trans Netw Service Manag https://doi.org/10.1109/TNSM.2024.3437217

[3] AlQahtani SA (2025) Dynamic slice allocation in 5G cloud RAN. Wireless Personal Communications 143(3-4):427–467. https://doi.org/10.1007/

s11277-025-11835-y

[4] Amiri E, Wang N, Shojafar M, et al (2024) Edge-AI empowered dynamic VNF splitting in O-RAN slicing: A federated DRL approach. IEEE Communications Letters 28(2):318–322. https://doi.org/10.1109/LCOMM.2023.3341302

[5] Bega D, Gramaglia M, Banchs A, et al (2020) A machine learning approach to 5G infrastructure market optimization. IEEE Transactions on Mobile Computing 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[6] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[7] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC), pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[8] Doanis P, Spyropoulos T (2024) Sample-efficient multi-agent DQNs for scalable multi-domain 5G+ inter-slice orchestration. IEEE Transactions on Machine Learning in Communications and Networking 2:956–977. https://doi.org/10.1109/TMLCN.2024.3420268

[9] Fatehi K, Rahmani Ghourtani M, Sonee A, et al (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: Proc. IEEE Int. Conf. Commun. (ICC), https://doi.org/10.1109/ICC59461.2026.11586913

[10] Gong Y, Gong P, Jiang H, et al (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG. China Commun 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[11] Hao X, She C, Yeoh PL, et al (2024) Hybrid-task meta-learning: A GNN approach for scalable and transferable bandwidth allocation. IEEE Transactions on Wireless Communications 23(12):19820–19835. https://doi.org/10.1109/TWC.2024.3487533

[12] Kamal YB, Abdulkafi AA (2026) Reinforcement learning double deep Q-network techniques for NextG network. Wireless Personal Communications 146(6):3175–3197. https://doi.org/10.1007/s11277-026-12048-7

[13] Khani M, Jamali S, Sohrabi MK, et al (2024) Slice admission control in 5G cloud radio access network using deep reinforcement learning: A survey. International Journal of Communication Systems 37(13):e5857. https://doi.org/10.1002/dac.5857

[14] Krishnamoorthy R, Gireesh Babu CN, Gite P, et al (2026) Federated learning for dynamic resource allocation in 6G network slicing. Wireless Personal Communications https://doi.org/10.1007/s11277-026-11943-3

[15] Kulkarni D, Venkatesan M, Kulkarni AV (2024) Actor critic based reinforcement learning for joint resource allocation and throughput maximization in 5G RAN slicing. Wireless Personal Communications 138(1):623–640. https://doi.org/10.1007/s11277-024-11526-0

[16] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: 2025 Multimedia University Engineering Conference (MECON), pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[17] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: 2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[18] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimization techniques for user-centric distributed network slicing in 5G networks. IEEE Access 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[19] Li F, Hao Y, Yang S, et al (2025) OACR^2: Online admission control and resource reservation for 5G slice networks with deep reinforcement learning. IEEE Transactions on Mobile Computing 24(8):7360–7376. https://doi.org/10.1109/TMC.2025.3548767

[20] Li Y, Wang H, Min G, et al (2026) A flexible and scalable multi-agent learning framework for dynamic RAN slicing in 6G native-AI networks. IEEE Transactions on Mobile Computing 25(5):7258–7273. https://doi.org/10.1109/TMC.2025.3642213

[21] Lim WYB, Luong NC, Hoang DT, et al (2020) Federated learning in mobile edge networks: A comprehensive survey. IEEE Communications Surveys & Tutorials 22(3):2031–2063. https://doi.org/10.1109/COMST.2020.2986024

[22] Ming Z, Yu H, Taleb T (2024) Federated deep reinforcement learning for prediction-based network slice mobility in 6G mobile networks. IEEE Transactions on Mobile Computing 23(12):11937–11953. https://doi.org/10.1109/TMC.2024.3404125

[23] Mohajer A, Mirzaei A, Darabi M, et al (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. IEEE Trans Netw Service Manag 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[24] Ojijo MO, Ramotsoela D, Oginga RA (2024) Slice admission control in 5G wireless communication with multi-dimensional state space and distributed action space: A sequential twin actor-critic approach. Computer Networks 255:110878. https://doi.org/10.1016/j.comnet.2024.110878

[25] Phyu HP, Naboulsi D, Stanica R (2025) ICE-CREAM: Multi-agent fully cooperative decentralized framework for energy efficiency in RAN slicing. IEEE Transactions on Network and Service Management 22(2):1859–1873. https://doi.org/10.1109/TNSM.2024.3524503

[26] Polese M, Bonati L, D'Oro S, et al (2023) ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms. IEEE Transactions on Mobile Computing 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[27] Polese M, Bonati L, D'Oro S, et al (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. IEEE Communications Surveys & Tutorials 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[28] Rashid T, Samvelyan M, Schroeder de Witt C, et al (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research 21(178):1–51

[29] Raza MR, Natalino C, Öhlen P, et al (2019) Reinforcement learning for slicing in a 5G flexible RAN. Journal of Lightwave Technology 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[30] Shao Y, Li R, Hu B, et al (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. IEEE Trans Veh Technol 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[31] Sirma M, Kavak A, Inner AB (2025) Optimizing relay selection in D2D communication for next-generation wireless networks using multi-agent reinforcement learning: A novel approach. Wireless Personal Communications 140(3-4):945–969. https://doi.org/10.1007/s11277-025-11753-z

[32] Sulaiman M, Moayyedi A, Ahmadi M, et al (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. IEEE Trans Netw Service Manag 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[33] Sunehag P, Lever G, Gruslys A, et al (2018) Value-decomposition networks for cooperative multi-agent learning based on team reward. In: Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS), pp 2085–2087, https://doi.org/10.65109/jsrc7365

[34] Tao Z, Xu W, You X (2025) Digital twin-accelerated online deep reinforcement learning for admission control in sliced communication networks. IEEE Transactions on Communications 73(4):2490–2504. https://doi.org/10.1109/TCOMM.2024.3476430

[35] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: Proc. Int. Wireless Commun. and Mobile Computing (IWCMC), https://doi.org/10.1109/IWCMC69287.2026.11580033

[36] Tsampazi M, D'Oro S, Polese M, et al (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: Proc. IEEE Global Communications Conference (GLOBECOM), pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[37] Tsampazi M, D'Oro S, Polese M, et al (2025) PandORA: Automated design and comprehensive evaluation of deep reinforcement learning agents for open RAN. IEEE Transactions on Mobile Computing 24(4):3223–3240. https://doi.org/10.1109/TMC.2024.3505781

[38] Wu Z, Pan S, Chen F, et al (2021) A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[39] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: Proc. IEEE VTS Asia Pacific Wireless Commun. Symp. (APWCS), https://doi.org/10.1109/APWCS67981.2025.11151868

[40] Yu C, Velu A, Vinitsky E, et al (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, pp 24611–24624, https://doi.org/10.52202/068431-1787

[41] Zangooei M, Golkarifard M, Rouili M, et al (2024) Flexible RAN slicing in Open RAN with constrained multi-agent reinforcement learning. IEEE Journal on Selected Areas in Communications 42(2):280–294. https://doi.org/10.1109/JSAC.2023.3336156

[42] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658